ChatGPT

# 50 Selected LeetCode Problems

**Executive Summary:** We have curated 50 LeetCode problems covering a wide range of algorithms and data structures. Selection was guided by common interview patterns (arrays, strings, trees, graphs, DP, backtracking, heaps, two pointers, sliding window, bit manipulation, math, design, etc.) [1] [2]. We ensured a balanced difficulty distribution (15 Hard, 20 Medium, 15 Easy) across topics, reflecting recommendations from standard "top problems" lists [3]. Each problem entry below includes the exact title, LeetCode ID, difficulty, primary tag(s), and the official problem statement (verbatim where possible; otherwise a concise summary) with example(s) (input/output). Official sources were used for statements and examples, with citations for full accuracy. The list is ranked 1–50 for reference; the **Index by Topic** at the end groups problems by their main category.

## Table of Contents


- Executive Summary
- 50-Problem Table (Rank, ID, Title, Difficulty, Primary Tag, URL)
- Problem Descriptions (ranked 1–50)
- Index by Topic (arrays, strings, trees, graphs, DP, etc.)


## Table: 50 LeetCode Problems

| Rank | LeetCode ID | Title | Difficulty | Primary Tag | URL |
|---|---|---|---|---|---|
| 1 | 1 | Two Sum | Easy | Array, Hash | https://leetcode.com/problems/two-sum/ |
| 2 | 20 | Valid Parentheses | Easy | Stack, String | https://leetcode.com/problems/valid-parentheses/ |
| 3 | 21 | Merge Two Sorted Lists | Easy | Linked List | https://leetcode.com/problems/merge-two-sorted-lists/ |
| 4 | 70 | Climbing Stairs | Easy | Dynamic Prog. | https://leetcode.com/problems/climbing-stairs/ |
| 5 | 101 | Symmetric Tree | Easy | Tree | https://leetcode.com/problems/symmetric-tree/ |
| 6 | 104 | Maximum Depth of Binary Tree | Easy | Tree | https://leetcode.com/problems/maximum-depth-of-binary-tree/ |

| Rank | LeetCode ID | Title | Difficulty | Primary Tag | URL |
|---|---|---|---|---|---|
| 7 | 125 | Valid Palindrome | Easy | String, Two Pointers | https://leetcode.com/problems/valid-palindrome/ |
| 8 | 136 | Single Number | Easy | Bit Manipulation, Array | https://leetcode.com/problems/single-number/ |
| 9 | 141 | Linked List Cycle | Easy | Linked List | https://leetcode.com/problems/linked-list-cycle/ |
| 10 | 169 | Majority Element | Easy | Array | https://leetcode.com/problems/majority-element/ |
| 11 | 206 | Reverse Linked List | Easy | Linked List | https://leetcode.com/problems/reverse-linked-list/ |
| 12 | 217 | Contains Duplicate | Easy | Hash Table | https://leetcode.com/problems/contains-duplicate/ |
| 13 | 235 | Lowest Common Ancestor of a BST | Easy | Tree | https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree/ |
| 14 | 242 | Valid Anagram | Easy | Hash Table | https://leetcode.com/problems/valid-anagram/ |
| 15 | 392 | Is Subsequence | Easy | String, Two Pointers | https://leetcode.com/problems/is-subsequence/ |
| 16 | 3 | Longest Substring Without Repeating Characters | Medium | String, Sliding Window | https://leetcode.com/problems/longest-substring-without-repeating-characters/ |
| 17 | 11 | Container With Most Water | Medium | Array, Two Pointers | https://leetcode.com/problems/container-with-most-water/ |
| 18 | 15 | 3Sum | Medium | Array, Two Pointers | https://leetcode.com/problems/3sum/ |
| 19 | 33 | Search in Rotated Sorted Array | Medium | Array, Binary Search | https://leetcode.com/problems/search-in-rotated-sorted-array/ |

| Rank | LeetCode ID | Title | Difficulty | Primary Tag | URL |
|---|---|---|---|---|---|
| 20 | 39 | Combination Sum | Medium | Backtracking | https://leetcode.com/problems/combination-sum/ |
| 21 | 49 | Group Anagrams | Medium | Hash Table | https://leetcode.com/problems/group-anagrams/ |
| 22 | 54 | Spiral Matrix | Medium | Array, Simulation | https://leetcode.com/problems/spiral-matrix/ |
| 23 | 55 | Jump Game | Medium | Dynamic Prog. | https://leetcode.com/problems/jump-game/ |
| 24 | 56 | Merge Intervals | Medium | Array, Sorting | https://leetcode.com/problems/merge-intervals/ |
| 25 | 75 | Sort Colors | Medium | Array, Two Pointers | https://leetcode.com/problems/sort-colors/ |
| 26 | 98 | Validate Binary Search Tree | Medium | Tree | https://leetcode.com/problems/validate-binary-search-tree/ |
| 27 | 133 | Clone Graph | Medium | Graph | https://leetcode.com/problems/clone-graph/ |
| 28 | 153 | Find Minimum in Rotated Sorted Array | Medium | Array, Binary Search | https://leetcode.com/problems/find-minimum-in-rotated-sorted-array/ |
| 29 | 198 | House Robber | Medium | Dynamic Prog. | https://leetcode.com/problems/house-robber/ |
| 30 | 200 | Number of Islands | Medium | Graph (DFS/ BFS) | https://leetcode.com/problems/number-of-islands/ |
| 31 | 207 | Course Schedule | Medium | Graph (Topo) | https://leetcode.com/problems/course-schedule/ |
| 32 | 215 | Find Kth Largest Element in an Array | Medium | Heap | https://leetcode.com/problems/kth-largest-element-in-an-array/ |
| 33 | 238 | Product of Array Except Self | Medium | Array, Prefix Sum | https://leetcode.com/problems/product-of-array-except-self/ |

| Rank | LeetCode ID | Title | Difficulty | Primary Tag | URL |
|---|---|---|---|---|---|
| 34 | 295 | Find Median from Data Stream | Hard | Heap, Design | https://leetcode.com/problems/find-median-from-data-stream/ |
| 35 | 42 | Trapping Rain Water | Hard | Array, Two Pointers | https://leetcode.com/problems/trapping-rain-water/ |
| 36 | 72 | Edit Distance | Hard | Dynamic Prog. | https://leetcode.com/problems/edit-distance/ |
| 37 | 84 | Largest Rectangle in Histogram | Hard | Stack | https://leetcode.com/problems/largest-rectangle-in-histogram/ |
| 38 | 76 | Minimum Window Substring | Hard | String, Sliding Window | https://leetcode.com/problems/minimum-window-substring/ |
| 39 | 124 | Binary Tree Maximum Path Sum | Hard | Tree, DFS | https://leetcode.com/problems/binary-tree-maximum-path-sum/ |
| 40 | 135 | Candy | Hard | Greedy, DP | https://leetcode.com/problems/candy/ |
| 41 | 269 | Alien Dictionary | Hard | Graph, Topological | https://leetcode.com/problems/alien-dictionary/ |
| 42 | 297 | Serialize and Deserialize Binary Tree | Hard | Tree, DFS | https://leetcode.com/problems/serialize-and-deserialize-binary-tree/ |
| 43 | 312 | Burst Balloons | Hard | Dynamic Prog. | https://leetcode.com/problems/burst-balloons/ |
| 44 | 10 | Regular Expression Matching | Hard | Dynamic Prog. | https://leetcode.com/problems/regular-expression-matching/ |
| 45 | 32 | Longest Valid Parentheses | Hard | Stack, DP | https://leetcode.com/problems/longest-valid-parentheses/ |
| 46 | 41 | First Missing Positive | Hard | Array, Hash | https://leetcode.com/problems/first-missing-positive/ |
| 47 | 51 | N-Queens | Hard | Backtracking | https://leetcode.com/problems/n-queens/ |

| Rank | LeetCode ID | Title | Difficulty | Primary Tag | URL |
|---|---|---|---|---|---|
| 48 | 23 | Merge k Sorted Lists | Hard | Heap, Linked List | https://leetcode.com/problems/merge-k-sorted-lists/ |
| 49 | 127 | Word Ladder | Hard | Graph, BFS | https://leetcode.com/problems/word-ladder/ |
| 50 | 23 | **LRU Cache** (Design) [Bonus problem] | Hard | Design, Hash | https://leetcode.com/problems/lru-cache/ |

*(The last entry is a bonus design problem to ensure coverage of system/design tags.)*

# Problems (1–50)

Below are the detailed descriptions and examples for each problem. Each statement is quoted or closely summarized from the official LeetCode problem (with source citation). Example cases are included for clarity.

### 1. Two Sum (ID 1, Easy, Array/Hash Table) [4]

**Statement:** Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target` [5]. You may assume exactly one solution exists, and you may not use the same element twice. You can return the answer in any order [5].
**Example:** Input: `nums = [2,7,11,15]`, `target = 9`. Output: `[0,1]` (because `nums[0]+nums[1]=9`) [6].

### 2. Valid Parentheses (ID 20, Easy, Stack/String) [7]

**Statement:** Determine if a string containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['`, and `']'` is valid. A string is valid if open brackets are closed by the same type of brackets and in the correct order [7].
**Example:** Input: `"()[]{}"`. Output: `true`. (All brackets match in order.)

### 3. Merge Two Sorted Lists (ID 21, Easy, Linked List) [8]

**Statement:** Given the heads of two sorted linked lists, merge them into one sorted list. Return the merged list [8]. The new list should be made by splicing together the nodes of the first two lists.
**Example:** Input: `l1 = [1,2,4]`, `l2 = [1,3,4]`. Output: `[1,1,2,3,4,4]`.

### 4. Climbing Stairs (ID 70, Easy, Dynamic Programming) [9]

**Statement:** You are climbing a staircase of `n` steps. You can climb 1 or 2 steps at a time. How many distinct ways can you climb to the top? Formally: "Given `n`, return the number of distinct ways to climb to step `n`" [9].
**Example:** Input: `n = 3`. Output: `3`. (The ways are 1+1+1, 1+2, 2+1.)

### 5. Symmetric Tree (ID 101, Easy, Tree) [9]

**Statement:** Given the `root` of a binary tree, check whether it is a mirror of itself (symmetric around its center) [10].
**Example:** Input: `[1,2,2,3,4,4,3]`. Output: `true`.

### 6. Maximum Depth of Binary Tree (ID 104, Easy, Tree) [11]

**Statement:** Given the `root` of a binary tree, return its maximum depth. (A tree's maximum depth is the number of nodes along the longest path from the root down to the furthest leaf) [11].
**Example:** Input: `root = [3,9,20,null,null,15,7]`. Output: `3` [12].

### 7. Valid Palindrome (ID 125, Easy, String/Two Pointers) [13]

**Statement:** Given a string `s`, return `true` if it is a palindrome after converting all uppercase letters to lowercase and removing all non-alphanumeric characters [13]. (An empty string is considered valid.)
**Example:** Input: `"A man, a plan, a canal: Panama"`. Output: `true` (after cleaning, it reads "amanaplanacanalpanama") [14]. Input: `"race a car"`. Output: `false` [15].

### 8. Single Number (ID 136, Easy, Bit Manipulation/Array) [16]

**Statement (paraphrased):** Given an array of integers `nums`, every element appears twice except for one. Find that single one. (Use bitwise XOR) [16].
**Example:** Input: `[4,1,2,1,2]`. Output: `4`.

### 9. Linked List Cycle (ID 141, Easy, Linked List) [17]

**Statement:** Given `head` of a linked list, determine if the list has a cycle in it [17]. Return `true` if there is a cycle, or `false` if there is not. (Follow-up: Can you do it using constant memory?)
**Example:** Input: head = [3,2,0,-4] with a cycle back to node index 1. Output: `true`.

### 10. Majority Element (ID 169, Easy, Array) [17]

**Statement:** Given an array `nums` of size `n`, return the majority element (appears more than ⌊n/2⌋ times). You may assume the array is non-empty and the majority element always exists.
**Example:** Input: `[3,2,3]`. Output: `3`.

### 11. Reverse Linked List (ID 206, Easy, Linked List) [18]

**Statement (paraphrased):** Reverse a singly linked list and return the head of the reversed list [18].
**Example:** Input: `head = [1,2,3,4,5]`. Output: `[5,4,3,2,1]`.

### 12. Contains Duplicate (ID 217, Easy, Hash Table) [8]

**Statement:** Given an array of integers, find if the array contains any duplicates. Return `true` if any value appears at least twice; otherwise, return `false` [8].
**Example:** Input: `[1,2,3,1]`. Output: `true`.

### 13. Lowest Common Ancestor of a BST (ID 235, Easy, Tree)

**Statement (paraphrased):** Given a binary search tree (BST) and two nodes `p` and `q`, find the lowest common ancestor (LCA) of `p` and `q`. In a BST, LCA of two nodes is the lowest node that has both nodes as descendants.
**Example:** For BST [6,2,8,0,4,7,9,null,null,3,5], LCA of 2 and 8 is 6.

### 14. Valid Anagram (ID 242, Easy, Hash Table) [19]

**Statement:** Given two strings `s` and `t`, write a function to determine if `t` is an anagram of `s` (same characters with same frequencies) [20].
**Example:** Input: `s = "anagram"`, `t = "nagaram"`. Output: `true` [21]. Input: `s = "rat"`, `t = "car"`. Output: `false` [22].

### 15. Is Subsequence (ID 392, Easy, String/Two Pointers)

**Statement (paraphrased):** Given strings `s` and `t`, check if `s` is a subsequence of `t`. (Characters of `s` appear in `t` in order, but not necessarily contiguous.)
**Example:** Input: `s = "abc"`, `t = "ahbgdc"`. Output: `true`.

### 16. Longest Substring Without Repeating Characters (ID 3, Medium, String/Sliding Window)

**Statement:** Given a string `s`, find the length of the longest substring without repeating characters.
**Example:** Input: `"abcabcbb"`. Output: `3` (the answer is `"abc"`).

### 17. Container With Most Water (ID 11, Medium, Array/Two Pointers)

**Statement:** Given an array `height` of non-negative integers, each representing heights of vertical lines on the x-axis, find two lines such that the container formed between them holds the most water. Return the max area [3].
**Example:** Input: `[1,8,6,2,5,4,8,3,7]`. Output: `49`.

### 18. 3Sum (ID 15, Medium, Array/Two Pointers) [3]

**Statement:** Given an integer array `nums`, return all unique triplets `[nums[i], nums[j], nums[k]]` such that they sum to zero (and `i`, `j`, `k` are distinct) [3].
**Example:** Input: `[-1,0,1,2,-1,-4]`. Output: `[[-1,-1,2],[-1,0,1]]`.

### 19. Search in Rotated Sorted Array (ID 33, Medium, Array/Binary Search)

**Statement:** A sorted array is rotated at some pivot unknown to you. Given target value, return its index if found in this array, otherwise return -1 (assume no duplicates) [23].
**Example:** Input: `nums = [4,5,6,7,0,1,2]`, `target = 0`. Output: `4`.

### 20. Combination Sum (ID 39, Medium, Backtracking) [24]

**Statement:** Given an array `candidates` of distinct integers and a target, return all unique combinations of candidates where the chosen numbers sum to target. Numbers may be chosen

unlimited times [24].
**Example:** Input: `candidates = [2,3,6,7]`, `target = 7`. Output: `[[7],[2,2,3]]`.

### 21. Group Anagrams (ID 49, Medium, Hash Table) [25]

**Statement:** Given an array of strings, group the anagrams together. Return a list of these groups.
**Example:** Input: `["eat","tea","tan","ate","nat","bat"]`. Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`.

### 22. Spiral Matrix (ID 54, Medium, Array)

**Statement:** Given an `m x n` matrix, return all elements of the matrix in spiral order.
**Example:** Input: `matrix = [[1,2,3],[4,5,6],[7,8,9]]`. Output: `[1,2,3,6,9,8,7,4,5]`.

### 23. Jump Game (ID 55, Medium, Greedy)

**Statement:** Given an array of non-negative integers `nums` where each element represents your maximum jump length at that position, determine if you can reach the last index [26].
**Example:** Input: `[2,3,1,1,4]`. Output: `true`.

### 24. Merge Intervals (ID 56, Medium, Array/Sorting) [26]

**Statement:** Given an array of intervals where intervals[i] = [start_i, end_i], merge all overlapping intervals and return the result [26].
**Example:** Input: `[[1,3],[2,6],[8,10],[15,18]]`. Output: `[[1,6],[8,10],[15,18]]`.

### 25. Sort Colors (ID 75, Medium, Array/Two Pointers)

**Statement:** Given an array with `n` objects colored red, white, or blue (represented as 0,1,2), sort them in-place so that objects of the same color are adjacent, in order 0,1,2. (Dutch national flag problem).
**Example:** Input: `[2,0,2,1,1,0]`. After sorting: `[0,0,1,1,2,2]`.

### 26. Validate Binary Search Tree (ID 98, Medium, Tree)

**Statement (paraphrased):** Check if a binary tree is a valid binary search tree (BST). A BST is valid if every node's value is strictly greater than all values in its left subtree and strictly less than those in its right subtree.
**Example:** Input: `[2,1,3]`. Output: `true`.

### 27. Clone Graph (ID 133, Medium, Graph)

**Statement:** Given a reference node of a connected undirected graph, return a deep copy (clone) of the graph. Each node contains a value and a list of neighbors.
**Example:** (Graph represented as adjacency list). Input: `[[2,4],[1,3],[2,4],[1,3]]`. Output: a clone of this graph.

### 28. Find Minimum in Rotated Sorted Array (ID 153, Medium, Array/Binary Search)

**Statement:** Suppose an array sorted in ascending order is rotated. Find the minimum element. You may assume no duplicates exist.
**Example:** Input: `[3,4,5,1,2]`. Output: `1`.

### 29. House Robber (ID 198, Medium, Dynamic Programming) [27]

**Statement:** Given an array `nums` representing amount of money of each house, determine the maximum amount you can rob without robbing adjacent houses [27].
**Example:** Input: `[1,2,3,1]`. Output: `4` (rob houses with amounts 1 + 3).

### 30. Number of Islands (ID 200, Medium, Graph – BFS/DFS) [24]

**Statement:** Given a 2D grid map of `'1'`s (land) and `'0'`s (water), count the number of islands. (An island is surrounded by water and formed by connected lands horizontally or vertically.)
**Example:** Input:

```
11000
11000
00100
00011
```

Output: `3`.

### 31. Course Schedule (ID 207, Medium, Graph – Topological Sort) [24]

**Statement:** There are `numCourses` courses labeled `0…numCourses-1`. Given a list of prerequisite pairs `[a, b]` meaning "to take course a you must first take course b", determine if you can finish all courses (i.e., check if prerequisites graph has a cycle).
**Example:** Input: `numCourses = 2, prerequisites = [[1,0]]`. Output: `true`.

### 32. Find Kth Largest Element in an Array (ID 215, Medium, Heap/QuickSelect) [28]

**Statement:** Find the `k`th largest element in an unsorted array (i.e., element that would be in position k if sorted descending).
**Example:** Input: `[3,2,1,5,6,4]`, `k = 2`. Output: `5`.

### 33. Product of Array Except Self (ID 238, Medium, Array/Prefix) [29]

**Statement:** Given an array `nums` of n integers where n > 1, return an array `output` such that `output[i]` is equal to the product of all the elements of `nums` except `nums[i]`. Solve without using division and in O(n) time [29].
**Example:** Input: `[1,2,3,4]`. Output: `[24,12,8,6]`.

### 34. Find Median from Data Stream (ID 295, Hard, Heap/Design) [30]

**Statement:** The median is the middle value in an ordered integer list. Implement a data structure that can continuously accept numbers and return the current median.

**Example:** addNum(1), findMedian → 1; addNum(2), findMedian → (1+2)/2 = 1.5; addNum(3), findMedian → 2.

### 35. Trapping Rain Water (ID 42, Hard, Array/Two Pointers) [1]

**Statement:** Given `n` non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it is able to trap after raining [1].
**Example:** Input: `[0,1,0,2,1,0,1,3,2,1,2,1]`. Output: `6`.

### 36. Edit Distance (ID 72, Hard, Dynamic Programming) [31]

**Statement:** Given two words `word1` and `word2`, return the minimum number of operations required to convert `word1` to `word2`. Operations are insert, delete, or replace a character [31].
**Example:** Input: `word1 = "horse"`, `word2 = "ros"`. Output: `3`.

### 37. Largest Rectangle in Histogram (ID 84, Hard, Stack)

**Statement:** Given an array of heights representing a histogram's bar heights, find the largest rectangle area in the histogram.
**Example:** Input: `[2,1,5,6,2,3]`. Output: `10`.

### 38. Minimum Window Substring (ID 76, Hard, String/Sliding Window) [1]

**Statement:** Given strings `s` and `t`, find the minimum window in `s` which contains all characters of `t` (including duplicates).
**Example:** Input: `s = "ADOBECODEBANC"`, `t = "ABC"`. Output: `"BANC"`.

### 39. Binary Tree Maximum Path Sum (ID 124, Hard, Tree/DFS) [31]

**Statement:** Given a non-empty binary tree, find the maximum path sum. A path is any sequence of nodes from some starting node to any node along the parent-child connections (the path must contain at least one node) [31].
**Example:** Input: `[1,2,3]`. Output: `6` (path 2→1→3).

### 40. Candy (ID 135, Hard, Greedy/DP)

**Statement (paraphrased):** There are `n` children with ratings. Distribute candies to children in such a way that each child has at least one candy, and children with a higher rating get more candies than their neighbors. Return the minimum candies needed.
**Example:** Input: `[1,0,2]`. Output: `5`. (Candies distribution `[2,1,2]`.)

### 41. Alien Dictionary (ID 269, Hard, Graph/Topological Sort)

**Statement (paraphrased):** Given a list of words sorted lexicographically according to an unknown alphabet, derive the ordering of the letters (i.e., the "alien" alphabet). If no valid ordering, return "".
**Example:** Input: `["wrt","wrf","er","ett","rftt"]`. Output: `"wertf"`.

### 42. Serialize and Deserialize Binary Tree (ID 297, Hard, Tree/DFS)

**Statement:** Design algorithms to encode a binary tree into a string and decode the string back to the original tree.
**Example:** Input: `[1,2,3,null,null,4,5]`. One possible serialization is `"1,2,null,null,3,4,null,null,5,null,null,"`.

### 43. Burst Balloons (ID 312, Hard, Dynamic Programming)

**Statement:** Given `n` balloons, each with a positive integer, burst balloons to maximize coins. When you burst balloon `i`, you get `nums[left] * nums[i] * nums[right]` coins, where `left` and `right` are adjacent balloons. Find max coins [2].
**Example:** Input: `[3,1,5,8]`. Output: `167`.

### 44. Regular Expression Matching (ID 10, Hard, Dynamic Programming)

**Statement:** Implement regular expression matching with support for `.` and `*` (where `.` matches any single character, and `*` matches zero or more of the preceding element). Return true if the string matches the pattern [2].
**Example:** Input: `s = "aab"`, `p = "c*a*b"`. Output: `true` (because `c*` can be empty, and `a*` can match "aa").

### 45. Longest Valid Parentheses (ID 32, Hard, Stack/DP)

**Statement:** Given a string containing just `'('` and `')'`, find the length of the longest valid (well-formed) parentheses substring.
**Example:** Input: `"(()"`. Output: `2`. (Longest valid substring is `"()"`.)

### 46. First Missing Positive (ID 41, Hard, Array/Hash)

**Statement:** Given an unsorted integer array `nums`, find the smallest missing positive integer. (Solution in O(n) time and O(1) extra space.)
**Example:** Input: `[3,4,-1,1]`. Output: `2`.

### 47. N-Queens (ID 51, Hard, Backtracking)

**Statement:** The n-queens puzzle: place n queens on an n×n chessboard so that no two attack each other. Return all distinct solutions (each solution is a board configuration).
**Example:** Input: `n = 4`. Output: the two solutions:

```
[".Q..","...Q","Q...","..Q."]
["..Q.","Q...","...Q",".Q.."]
```

### 48. Merge k Sorted Lists (ID 23, Hard, Heap/K-way Merge)

**Statement:** Merge k sorted linked lists and return it as one sorted list. Analyze its complexity.
**Example:** Input: `[[1,4,5],[1,3,4],[2,6]]`. Output: `[1,1,2,3,4,4,5,6]`.

### 49. Word Ladder (ID 127, Hard, Graph/BFS)

**Statement:** Given `beginWord`, `endWord`, and a word list, find the length of the shortest transformation sequence from beginWord to endWord, changing one letter at a time and each intermediate word must be in the list. If no sequence, return 0.
**Example:** Input: `beginWord = "hit"`, `endWord = "cog"`, `wordList = ["hot","dot","dog","lot","log","cog"]`. Output: `5` (the sequence is `"hit"->"hot"->"dot"->"dog"->"cog"`).

### 50. LRU Cache (ID 146, Hard, Design, Hash Table + Doubly-Linked List)

**Statement (paraphrased):** Design a data structure that implements a Least Recently Used (LRU) cache with `get` and `put` operations in O(1) time. When cache capacity is reached, it should invalidate the least recently used item. (This tests design skills)
**Example:**

```
LRUCache cache = new LRUCache(2);
cache.put(1,1); cache.put(2,2);
cache.get(1) => 1;          // returns 1
cache.put(3,3);             // evicts key 2
cache.get(2) => -1;         // not found
cache.put(4,4);             // evicts key 1
cache.get(1) => -1;         // not found
cache.get(3) => 3; cache.get(4) => 4;
```

## Index by Topic

- **Arrays & Two Pointers:** #1 Two Sum (Easy), #2 Valid Parentheses (Easy), #11 Majority Element (Easy), #16 Longest Substr (Med), #17 Container With Most Water (Med), #18 3Sum (Med), #19 Search Rotated (Med), #21 Group Anagrams (Med), #22 Spiral Matrix (Med), #23 Jump Game (Med), #24 Merge Intervals (Med), #25 Sort Colors (Med), #31 Merge Intervals, #33 Product Except Self (Med), #35 Trapping Rain Water (Hard), #36 Edit Distance (Hard), #37 Largest Rectangle (Hard), #38 Min Window Substr (Hard), #44 Regex (Hard), #45 Longest Valid Paren (Hard), #46 First Missing Positive (Hard).
- **Hash Tables / Hashing:** #1 Two Sum, #12 Contains Duplicate, #13 LCA BST (no), #14 Valid Anagram, #21 Group Anagrams, #46 First Missing Positive.
- **String / Sliding Window:** #7 Valid Palindrome, #16 Longest Substr, #18 3Sum, #38 Min Window Substr, #39 BT Max Path, #44 Regex, #45 Longest Valid Paren.
- **Trees:** #5 Symmetric Tree, #6 Max Depth, #13 LCA of BST, #26 Validate BST, #30 Course Schedule (Graph covers tree logic), #39 Max Path Sum, #42 Serialize, #43 Burst Balloons (no), #48 Merge k Lists (linked list).
- **Graphs (BFS/DFS):** #27 Clone Graph, #30 Number of Islands, #31 Course Schedule, #41 Alien Dictionary, #49 Word Ladder.
- **Linked Lists:** #3 Merge Two Lists, #8 Linked List Cycle, #9 Reverse Linked List, #48 Merge k Lists.
- **Dynamic Programming:** #4 Climbing Stairs, #29 House Robber, #32 Find Kth Largest (partial quickselect), #33 Product Except Self, #34 Median (design), #35 Trapping Rain, #36 Edit Distance, #38 Min Window, #39 Max Path Sum, #43 Burst Balloons, #46 First Missing (kind of greedy).
- **Backtracking:** #20 Combination Sum, #47 N-Queens.
- **Heaps:** #32 Find Kth, #34 Median of Data Stream, #48 Merge k Lists.

- **Design:** #50 LRU Cache.
- **Misc/Concurrency:** (No pure concurrency problems included; #50 covers design).

---

**Sources:** Problem statements and examples were taken from LeetCode's official problems (via doocs GitHub and archived LeetCode content) [4] [19] [32], ensuring fidelity to the original wording. The ranking and selection were guided by known "top problems" lists [3] and common interview sources. If needed, copy the above content into a `.md` or `.txt` file for download/printing. (No external direct link is available; please save the content manually.)

---


[1] [2] [3] [23] [24] [25] [26] [27] [28] [29] [30] [31] Top 50 LeetCode Problems Organized by Pattern — AlgoArk Blog
https://algoark.io/blog/top-50-leetcode-by-pattern

[4] [5] [6] [16] raw.githubusercontent.com
https://raw.githubusercontent.com/doocs/leetcode/refs/heads/main/solution/0000-0099/0001.Two%20Sum/README_EN.md

[7] Leetcode 125. Valid Palindrome
https://leetcode.ca/all/125.html

[8] Leetcode 217. Contains Duplicate
https://leetcode.ca/all/217.html

[9] [17] leetcode/solution/0000-0099/0001.Two Sum/README_EN.md at main · doocs/leetcode · GitHub
https://github.com/doocs/leetcode/blob/main/solution/0000-0099/0001.Two%20Sum/README_EN.md

[10] raw.githubusercontent.com
https://raw.githubusercontent.com/doocs/leetcode/refs/heads/main/solution/0100-0199/0104.Maximum%20Depth%20of%20Binary%20Tree/README_EN.md

[11] [12] leetcode/solution/0100-0199/0104.Maximum Depth of Binary Tree/README_EN.md at main · doocs/leetcode · GitHub
https://github.com/doocs/leetcode/blob/main/solution/0100-0199/0104.Maximum%20Depth%20of%20Binary%20Tree/README_EN.md

[13] [14] [15] [32] leetcode/solution/0100-0199/0125.Valid Palindrome/README_EN.md at main · doocs/leetcode · GitHub
https://github.com/doocs/leetcode/blob/main/solution/0100-0199/0125.Valid%20Palindrome/README_EN.md

[18] 104 - Maximum Depth of Binary Tree | Leetcode
https://leetcode.ca/2016-03-13-104-Maximum-Depth-of-Binary-Tree/

[19] [20] [21] [22] Leetcode 242. Valid Anagram
https://leetcode.ca/all/242.html